

■ Bug fix

+ degeneracy guard

```
trial = selected + [cand_idx]
cur_min = min_area(lattice[trial])
+ if cur_min <= 1e-12:
+     continue
    if cur_min > best_min:
        best_min = cur_min
```

■ External dependency

+ `scipy.spatial.ConvexHull`

```
import numpy as np
+ from scipy.spatial import ConvexHull
...
- return min_area(pts) / hull_const
+ hull = ConvexHull(pts)
+ return min_area(pts) / hull.volume
```

■ Architectural change

14-gon → 2 concentric 7-gons

```
- # regular 14-gon
- angles = np.arange(14) * 2*pi/14
- pts = R * stack((cos a, sin a))
+ # two concentric heptagons
+ outer = R * heptagon(angles_7)
+ for f in np.arange(.20,.91,.01):
+     inner = f*R*heptagon(angles_7+pi/7)
+     pts = vstack((outer, inner))
```

■ Composition

hill-climb + SA acceptance

```
+ start_temp = max(cur*0.5, 1e-8)
  for step in range(n_iters):
+     temp = start_temp*(1 - step/n_iters)
      if cand > cur:
          cur = cand
+     else:
+         p = exp((cand - cur)/temp)
+         if rng() < p: cur = cand
```

■ Local refinement

unit radius → unit area

```
angles = linspace(0, 2*pi, n)
points = stack((cos(a), sin(a)))
+ # scale to unit hull area
+ hull_area = (n/2)*sin(2*pi/n)
+ points *= sqrt(1.0 / hull_area)
  return points
```

■ Pruning

-48 lines, delegate to baseline

```
- cols = ceil(sqrt(n))
- radius = min(w, h) / (2*cols)
- centers, radii = [], []
- # ... 40+ lines of placement
- return centers, radii, sum_radii
+ return baseline_packing(instance)
```

■ Refactor

extract helper function

```
- def heilbronn_convex14():
-     # 80-line monolithic body
-     ...
+ def _heptagon_layout():
+     ... # extracted
+ def heilbronn_convex14():
+     return _heptagon_layout()
```

■ Efficiency

hoist `combinations()` out of loop

```
+ COMBOS = np.array(combinations(N, 3))

  def min_area(pts):
-     idx = np.array(combinations(N,3))
-     p1,p2,p3 = pts[idx[:,0]], ...
+     p1,p2,p3 = pts[COMBOS[:,0]], ...
```

■ Hyperparameter tuning

single literal change

```
// solver budget
- double time_limit = 1.85;
+ double time_limit = 1.90;
  solver.set_time_limit(time_limit);
```